

12_helixqa_testing

Extending HelixQA to Fully Test HelixLLM Capabilities (esp. Vision) — Cited Research + Design

Doc ID: 12_helixqa_testing **Date:** 2026-07-06 **Author:** T1 deep-research + design subagent **Scope:** Design a HelixQA extension that tests every HelixLLM capability — LLM chat/tool-calling/streaming, **vision/VLM**, image-gen, video-gen, translation, embeddings/RAG, STT, OCR, provider-coverage — with rock-solid captured proofs and zero bluff, wired into autonomous QA sessions. **Governance:** §11.4.107 (liveness/freeze/frame-advance/self-validated analyzers), §11.4.117 (CV/OCR pixel-oracle), §11.4.137 (content-correctness oracle), §11.4.158–160 (intensive window-scoped recording + read-the-screen vision validation + HelixQA bridge + media-validation pipeline), §11.4.163 (universal media validation), §11.4.116 (real-time sync channel), §11.4.98 (full-autonomous, no manual step), CONST-050(B) (all test types), §11.4.6 (no-guessing — every mechanism below is quoted from source).

§11.4.6 honesty boundary — HelixLLM contract UNCONFIRMED. No single canonical HelixLLM service spec was found in the tree (`grep -rile "helixllm|helix_llm"` returns only i18n bundles and generic app files, not an API contract). This design therefore targets the **standard capability set** and the **existing HelixCode LLM/provider surface** (`helix_code/internal/{llm,provider,providers,verifier}`, the `/api/v1/llm/generate` endpoint in `root CLAUDE.md §9`, the `LLMsVerifier` `CONST-036/039` provider list). **Before implementation, the exact HelixLLM endpoint paths, request/response schemas, and which capabilities ship must be confirmed** and the bank `http/dispatches_to` values filled from that contract. All bank YAML below uses placeholder paths marked `# CONFIRM`.

Part 1 — HelixQA’s ACTUAL Mechanisms (read from submodules/helix_qa, §11.4.6)

HelixQA is an **anti-bluff QA orchestration framework** (`README.md`): “the bar for shipping is not *tests pass* but *users can use the feature*. Every PASS HelixQA emits

MUST carry positive runtime evidence.” Five concrete mechanisms are load-bearing for this design.

1.1 Test-bank format — `pkg/testbank/schema.go`

YAML banks in `banks/*.yaml` load into `TestCase` structs. The user-facing fields are `id`, `name`, `description`, `category`, `priority`, `platforms`, `steps[]`, `tags`, `documentation_refs`, `estimated_duration`, `expected_result`. The **anti-bluff** / **executable** fields (the ones this design leans on) are:

- **Steps[] .Action typed by ActionType** (executable, not prose): `http` ("`<METHOD> <PATH>`" + `Body/Headers/ExpectStatus/ExpectJSONPath/ExpectBodyContains/AuthMode`), `shell`, `adb_shell`, `playback_check` (`dumpsys media_session state==3`), `frame_diff` (`screenshot` → `wait` → `screenshot`, fail if frozen), `assert` (`status_eq/json_path_eq/body_contains/header_eq`), `screenshot/tap/keypress/sleep`. The comment on `ActionTypeHTTP` records that prose-only description actions were a real bluff (“4034 PROSE_HELIXQA_ACTION findings”) — **so a new bank MUST use executable action types, never prose**.
- **DispatchesTo string** — names a consumer on-device/host script that IS the Challenge body; its **real exit code drives PASS/FAIL** via the consumer-injected `DeviceExec` hook.
- **RequiredEvidence []string** — the §11.4.69 evidence-ledger gate: globs/keys that must each resolve to a real, non-empty artefact (or a conduit `evidence_captured` event with non-empty path) before PASS. Missing → FAIL with the list.
- **Metadata map[string]any** — opaque consumer data; the loader preserves it verbatim. Used e.g. for `metadata.recvalidate_options`.
{`reply_markers`, `chrome_line_patterns`, `expected_replies`}
forwarded to the recording oracle.
- `feature_class` (maps to the §11.4.69 taxonomy), `domains[]` (single-resource-owner partitioning), `requires_env[]` (honest SKIP-OK when hardware/keys absent), `challenge_id`.

1.2 The evidence oracle — `pkg/testbank/content_evidence.go` **ContentAssertingResolver**

This is the key anti-bluff primitive. `GlobEvidenceResolver` only checks a file exists + `Size()>0` — a “non-empty-but-WRONG file (`echo stereo > codec.txt`)” bluff-PASSes. `ContentAssertingResolver` upgrades a `RequiredEvidence` token to a generic **declarative assertion grammar**, project-agnostic (HelixQA hardcodes no value; the bank supplies the regex/key/threshold):

```

<path-or-glob> | <assertion> [ | <assertion> ... ]
  nonempty
  match:<regex>                (case-insensitive)
  not_match:<regex>
  json:<dotted.path><op><val>    ops: == != >= <= > <    e.g. json:video_li
  min_int:<field>:<n>           first int after <field> token >= n

```

Every capability bank below expresses its correctness assertion as a RequiredEvidence token in this grammar — that is the seam that makes a PASS mean the produced artefact is *right*, not merely present.

1.3 Dispatcher scoring — pkg/testbank/dispatch.go

Dispatcher.Run

Sequence: (1) run DispatchesTo via DeviceExec; non-zero exit → **FAIL**; (2) enforce the RequiredEvidence ledger via the resolver; any missing → **FAIL** even if the script exited 0 (closes the “green exit, no proof” hole); (3) else **PASS**, citing satisfied evidence. Every verdict folds into a conduit challenge_verdict event. requires_env unmet → SKIP.

1.4 Real-time sync channel — pkg/conduit (§11.4.116)

Dependency-free, two transports: an **append-only JSONL event stream** (one Event per line, Seq-numbered, tailed by Monitor like tail -f) + an **atomically-overwritten status snapshot** (Status with live Counts). Closed event set: session_start/end, phase_start/complete/error/progress, challenge_start/step/verdict, **evidence_captured (carries evidence_path+evidence_kind)**, **llm_call**, **vision_call**, error, log. Verdict vocabulary: PASS/FAIL/SKIP/OPERATOR-BLOCKED. A **verdict event’s backing evidence is auditable by the conductor** — a PASS challenge_verdict with no preceding evidence_captured for that challenge is a contradiction (§11.4.116). llm_call/vision_call already exist as first-class events — the LLM/vision banks light them up natively.

1.5 Vision + media toolchain (already shipped)

- **pkg/vision**: OllamaClient (LLaVA VLM: AnalyzeImage(img, prompt) → structured UIAnalysisResult; CheckOllamaAvailable, GetAvailableModels, PullModel); **Tesseract OCR** (ocr_tesseract.go: DetectText/DetectTextString, GetAvailableLanguages, per-word TSV with confidence); **PaddleOCR** (ocr_paddle.go); **ElementDetector** (CV contours); diff.go/perceptual//hash//template/ (freeze / frame-advance / perceptual-hash — the §11.4.107 liveness primitives).
- **pkg/recordingqa** (recordingqa.go): the **media-validation orchestrator** (§11.4.163). PASS **only if** mp4 non-empty **AND** injected VideoValidator

(Panoptic recvalidate / recording-analyzer) matches every ExpectedReply with NO error text **AND** stderr log has NONE of the error patterns. VideoOptions{ExpectedReplies, ChromeLinePatterns, ReplyMarkers} excludes chrome-as-reply (§11.4.137). Ships **golden-good + golden-bad + paired \$1.1 mutation** tests (recordingqa_test.go: dropping chrome forwarding makes the golden-bad bluff-PASS → mutation proves the guard is real).

- **cmd/ analyzers:** recording-analyzer (frame OCR + §11.4.107 liveness), helixqa-recvalidate, helixqa-omniparser, helixqa-uitars, helixqa-lpips, helixqa-dreamsim (perceptual/full-reference metrics), qa-audio-probe, helixqa-{x11grab, kmsgrab} (window-scoped capture, §11.4.159), helixqa-conduit-monitor.
- **Autonomous session:** pkg/autonomous (SessionCoordinator, PlatformWorker, PhaseManager, pipeline.go, http_executor.go, structured_executor.go, findings_bridge.go); cmd/helixqa run|autonomous|bank-session.

Gap this design fills: HelixQA has the vision/OCR/liveness/recording/evidence/conduit machinery and generic banks, but has **no HelixLLM-capability banks** — no bank that sends an image to a VLM and self-validates the answer, no bank that validates image/video-gen output is real media, no translation/embeddings/RAG/STT banks. All new work is bank YAML + a thin per-capability dispatches_to analyzer that emits a JSON verdict the ContentAssertingResolver gates on — **no core-engine change**, per CONST-051(B) decoupling.

Part 2 — Best-Practice Research (cited; today 2026-07-06)

(a) Local LLM serving endpoint — correctness, tool-calling, streaming, concurrency/load

Two orthogonal test classes: **correctness** (deterministic-answerable prompts, assert exact/normalized match — e.g. word-number → digit) and **load** (Gatling, ray-project **LLMPerf**, NVIDIA GenAI-Perf, vLLM bench, Locust+SSE). Streaming must test the **SSE** path: **TTFT** (time-to-first-token) is the dominant user-perceived metric; measure inter-token latency, output-tokens/s, **p99 TTFT under concurrency**, 5xx-rate-at-spike. Error paths (429/503/timeout, truncated output) must themselves be load-tested. Realistic traffic varies prompt length/domain — never identical short prompts. [Gatling; LLMPerf; LoadForge; PremAI 2026]

(b) VLM / vision endpoint (send image → assert correct understanding)

VLM-as-a-judge and automated pipelines (Auto-Bench, Stanford VHELM's nine aspects) are the state of the art. Practical automated signals: **image-text alignment via CLIP cosine >0.8 = strong**; **hallucination rate 10–30% typical on complex scenes**; spatial-reasoning accuracy 50–60%. Key anti-bluff lesson: measure against **ground-truth Q/A pairs** where the answer is knowable, and **self-validate the analyzer** with a golden-good/golden-bad pair (§11.4.107(10)) so the judge itself can't bluff. [Stanford VHELM; Nature s41598-026-55179-4; arXiv 2311.14580 LLM-as-Aligner; Label Your Data 2026]

(c) Image / video generation (validate output is REAL media, not a stub)

Image: **CLIP score** (prompt adherence), **FID** (realism vs a real-image distribution, lower=better) — combine automated metrics with periodic human validation. Video: **ffprobe -select_streams v:0 -count_frames -show_entries stream=nb_read_frames** for real decoded-frame count; assert non-zero frames + expected codec/resolution/fps + **frame-advance** (not a frozen single frame, §11.4.107). [ffmpeg.org/ffprobe; OTTVerse; Labelbox; ImageBench prompt-fidelity]

(d) Translation quality (COMET / back-translation / metamorphic)

COMET neural metric correlates >0.8 with human judgment, beating BLEU; **COMET-QE** referenceless exploits source ↔ target symmetry (back-translation intuition). **Back-translation metamorphic check**: translate $X \rightarrow Y \rightarrow X'$, assert semantic similarity(X, X') high — no reference needed, fully autonomous. WMT25 ESA quality-estimation is the current shared-task baseline. [Grokipedia MT-eval; arXiv 2508.18549 COMET-poly; TACL Conformalizing MT Eval]

(e) Embeddings / RAG (retrieval quality)

Retrieval: **Recall@K** (most actionable), **MRR** (rank-1 precision), **nDCG** (correlates best with end-to-end RAG quality). Generation: **faithfulness** (proportion of answer claims verifiable against retrieved chunks — RAGAS LLM-as-judge), answer-relevancy, context-relevancy. Version index/embeddings/rerankers/eval-sets; **index-integrity test** (embeddings stored + queryable). [langcopilot RAG-101; Confident AI; getmaxim 2025; Label Your Data]

(f) STT / OCR

STT: **WER** (word-level) + **CER** (char-level; CER for CJK where word boundaries are ambiguous); Whisper-Large-v3 ≈ 2.7% WER clean / 8–12% real-world (MLPerf v5.1)

reference). OCR: **CER** (edit-distance/GT-chars) primary; printed-text 98–99% is excellent, 2025 avg ≈96.5%; per-word confidence floor + ROI to avoid false-pos/neg (§11.4.137(12)). [MLCommons Whisper v5.1; whisperapi WER; TowardsDataScience CER/WER; HandwritingOCR CER]

Part 3 — HelixQA Extension: Per-Capability Test Banks

Design pattern (uniform, anti-bluff, decoupled). Each capability gets a bank `banks/helixllm_<capability>.yaml`. Each `TestCase`: 1. drives the **real HelixLLM endpoint** (`http` step or a `dispatches_to` analyzer that calls it), 2. writes a machine-readable **verdict artefact** (`*_verdict.json`) capturing the metric + the raw response + the ground-truth, 3. gates **PASS** on a **RequiredEvidence content assertion** (§1.2 grammar) over that artefact — so **PASS** = the metric passed, not “a file exists”, 4. sets `feature_class` for the §11.4.69 taxonomy, `requires_env` for keys/hardware (honest **SKIP**), and (media capabilities) `metadata.recvalidate_options` for the §11.4.163 pipeline, 5. ships a **paired §1.1 mutation** (a golden-bad fixture the analyzer must **FAIL**).

Each capability’s analyzer is a small **cmd/helixqa-verify-<cap>** Go tool (consumer-injected `DeviceExec`), reusing existing `pkg/vision` (VLM/OCR), `pkg/recordingqa`, and `stdlib` — **no core change**.

Bank A — `helixllm_llm.yaml` (chat correctness + tool-calling + streaming + concurrency)

- **LLM-CORRECT-001** deterministic-answer prompts (arithmetic, word → digit, JSON-shape). `http POST <llm/generate> #CONFIRM`;
`RequiredEvidence: "qa/llm/correct_001_verdict.json | json:exact_match==true"`. Anti-bluff: prompt has **ONE** correct answer; analyzer normalizes then compares; **no LLM-judge for objectively-checkable answers** (avoids judge-bluff).
- **LLM-TOOL-001** tool-calling: send a prompt that must emit a tool call with specific args; assert the returned tool-call name+args JSON exactly. . . . | `json:tool.name==get_weather | json:tool.args.city==Belgrade`.
- **LLM-STREAM-001** SSE streaming: analyzer opens the stream, records **TTFT**, inter-token gaps, total tokens; asserts `json:ttft_ms<=<budget> + json:token_count>=2 + json:monotonic_stream==true` (proves genuine incremental delivery, not one buffered blob). `feature_class: llm_stream`.
- **LLM-CONCURRENCY-001** (stress, §11.4.85): $N \geq 10$ parallel generations via `LLMPerf`-style driver; assert `json:p99_ttft_ms<=<budget> +`

json:error_rate==0 + json:deadlocks==0. domains:[llm] single-owner if endpoint is exclusive.

- **LLM-ERROR-001** inject over-limit/429 and assert graceful typed error, not a crash/silent-empty.
- **Mutation:** a fixture where the model returns a wrong digit $\Rightarrow$ exact_match analyzer must emit false $\Rightarrow$ gate FAILs.

Bank B — helixllm_vision.yaml (VLM — THE headline capability)

Uses pkg/vision.OllamaClient **or** the HelixLLM vision endpoint. Ground-truth image corpus committed under data/vision_gt/ with per-image expected facts.

- **VIS-UNDERSTAND-001** send a known image (data/vision_gt/red_stop_sign.png) + prompt “What shape and color is the sign?”; analyzer checks the VLM answer contains the GT tokens $\rightarrow$ qa/vision/understand_001_verdict.json | json:matched_facts==2 | json:hallucinated==false. Emits a vision_call conduit event. feature_class: video_display/image_understanding. - **VIS-COUNT-001** / **VIS-OCR-READ-001** / **VIS-SPATIAL-001** counting (“how many cats”), text-in-image reading (cross-check VLM answer against **Tesseract OCR** of the same image — two-oracle agreement), spatial (“is the red box left of the blue?”). Each RequiredEvidence asserts the GT via json: or match:.. - **VIS-CLIP-001** (optional, if a CLIP embed endpoint exists) assert image-text json:clip_cosine $\geq$ 0.8 for aligned pair AND <0.5 for a deliberately mismatched pair (metamorphic). - **VIS-SELF-VALIDATE-001 (mandatory §11.4.107(10))**: run the analyzer on a **golden-good** pair (answer correct $\rightarrow$ must PASS) AND a **golden-bad** pair (VLM given a mismatched GT $\rightarrow$ must FAIL). The paired mutation strips the fact-matching assertion and asserts the golden-bad then bluff-PASSes — proving the vision oracle itself cannot bluff. This is the single most important new test. - **VIS-LIVENESS-001** for any “describe the live screen” flow: window-scoped capture (helixqa-x11grab, §11.4.159) $\rightarrow$ freeze-detection (pkg/vision/diff) $\rightarrow$ assert not-stale before the VLM judges it.

Bank C — helixllm_imagegen.yaml

- **IMG-GEN-001** prompt $\rightarrow$ image; analyzer asserts (1) real image bytes decode (image.Decode ok, non-trivial dimensions — not a 1 $\times$ 1 stub), (2) **CLIP prompt-adherence** json:clip_score $\geq$ <cal>, (3) optional **FID** vs a reference set. RequiredEvidence: "qa/imagegen/gen_001.png | nonempty" , "qa/imagegen/gen_001_verdict.json | json:decodes==true | json:clip_score $\geq$ 0.25". Thresholds **calibrated on our own fixtures** (§11.4.6), not literature-copied.
- **Mutation:** feed the analyzer a solid-gray/blank PNG $\Rightarrow$ CLIP-adherence FAILs (proves it rejects stubs).

Bank D — helixllm_videogen.yaml

- **VIDGEN-001** prompt → video; analyzer runs **ffprobe -count_frames** → `json:nb_frames>=<min>` + expected codec/width/height/fps + **frame-advance** over the clip (perceptual-hash distance between sampled frames > ϵ → not a frozen loop, §11.4.107) + first-frame≠last-frame not-stale. Feeds pkg/recordingqa (§11.4.163). `feature_class: mediacodec_decode`.
- **Mutation:** a 0-frame / single-repeated-frame mp4 ⇒ frame-advance FAILs.

Bank E — helixllm_translation.yaml

- **TR-BT-001 metamorphic back-translation** (no reference needed, fully autonomous): $X(\text{en}) \rightarrow Y(\text{sr}) \rightarrow X'(\text{en})$; analyzer computes semantic similarity(X, X') (embedding cosine or **COMET-QE**) → `json:round_trip_sim>=<cal>`.
- **TR-COMET-001** where a reference exists: `json:comet>=<cal>`.
- **TR-INVARIANT-001** metamorphic invariants (numbers/named-entities/URLs preserved verbatim across translation) → `match: on entity preservation`.
- **Mutation:** identity “translation” (returns source unchanged) ⇒ back-translation sim is trivially 1.0 but the entity/target-language check (`json:target_lang==sr`) FAILs — proving BT alone isn’t enough and the guard catches a no-op translator.

Bank F — helixllm_embeddings_rag.yaml

- **EMB-INTEGRITY-001** embed N texts; assert vectors stored + queryable + stable dim (`json:dim==<D>` + `json:nan_count==0`).
- **EMB-SIM-001** metamorphic: $\text{sim}(\text{paraphrase-pair}) > \text{sim}(\text{unrelated-pair})$ → `json:relative_order_ok==true`.
- **RAG-RECALL-001 / RAG-MRR-001 / RAG-NDCG-001** over a labelled $Q \rightarrow \text{relevant-doc set}$ → `json:recall_at_5>=<cal>`, `json:mrr>=<cal>`, `json:ndcg>=<cal>`.
- **RAG-FAITHFULNESS-001** RAGAS-style: `json:faithfulness>=<cal>` (fraction of answer claims grounded in retrieved chunks) + a **negative** case (answer with an unsupported claim → faithfulness must drop).
- **Mutation:** shuffle the retrieval labels ⇒ Recall@K collapses ⇒ gate FAILs.

Bank G — helixllm_stt.yaml

- **STT-WER-001** known-transcript audio (data/stt_gt/*.wav + .txt); analyzer computes **WER/CER** vs GT → `json:wer<=<cal>` (`json:cer<=<cal>` for CJK). qa-audio-probe confirms the WAV is non-silent (RMS floor) first (§11.4.68). `requires_env: [HELIXQA_LAB_HAS_AUDIO_GT]`.

- **Mutation:** feed silence/empty transcript ⇒ WER=100% ⇒ FAIL.

Bank H — `helixllm_ocr.yaml`

- **OCR-CER-001** GT-text images; **Tesseract/Paddle** (pkg/vision) → **CER** vs GT → `json:cer<=<cal> + per-word json:min_confidence>=<floor> + ROI` (§11.4.137(12)). Multi-language via `GetAvailableLanguages`.
- **Mutation:** blank/noise image asserted against real GT ⇒ CER high ⇒ FAIL.

Bank I — `helixllm_providers.yaml` (CONST-036/039 provider-coverage)

- **PROV-COVER-001** for each verifier-supported provider (OpenAI, Anthropic, Gemini, DeepSeek, Groq, Mistral, xAI, OpenRouter, Ollama, Llama.cpp): a minimal live generate + one capability probe. `requires_env:` [`<PROVIDER>_API_KEY`] → honest **SKIP** when the key is absent (never a faked PASS, §11.4.98/§11.5). Cross-checks the model list is verifier-sourced, not hardcoded (CONST-036).
-

Part 4 — Anti-Bluff Captured-Evidence Spec (applies to every bank)

1. **PASS requires a content assertion, never file-exists.** Every `RequiredEvidence` token carries at least one `json:/match:/min_int:` clause over the verdict artefact (§1.2). A bare nonempty PASS is forbidden for a correctness claim.
2. **The verdict artefact embeds the raw response + the ground-truth + the metric**, so the conductor (and a human) can recompute it — the analyzer's own conclusion is auditable, not trusted.
3. **Self-validated analyzers (§11.4.107(10)) are mandatory** for VLM, image-gen, video-gen, translation, embeddings, STT, OCR: each ships a golden-good (must PASS) + golden-bad (must FAIL) fixture pair, wired into the meta-test; the **paired §1.1 mutation** strips the discriminating assertion and asserts the golden-bad then bluff-PASSes.
4. **Conduit evidence linkage (§11.4.116):** every PASS `challenge_verdict` is preceded by an `evidence_captured` event whose `evidence_path` is the verdict artefact; `llm_call/vision_call` events carry model + latency + token counts in `Fields`. A PASS with no backing `evidence_captured` is a channel-layer bluff → treated as FAIL.
5. **Liveness for any on-screen judgement (§11.4.107/159):** window-scoped capture only, freeze-detection + frame-advance before a VLM/OCR reads a live surface; not-stale cross-check.

6. **Media validation (§11.4.163):** video/image/recording artefacts flow through pkg/recordingqa + ffprobe/recording-analyzer; a missing tool surfaces a typed ToolAbsentError → honest **SKIP-with-reason**, never a fake PASS.
 7. **Honest SKIP, never fake PASS:** absent key/hardware/GT ⇒ requires_env SKIP-OK; genuinely infeasible autonomous path ⇒ OPERATOR-BLOCKED (§11.4.52), tracked, never green.
 8. **Thresholds calibrated on our own fixtures (§11.4.6),** recorded in the bank next to the assertion — never copied from a paper.
-

Part 5 — Autonomous-Session “keep-it-running + live-testable” Acceptance (§11.4.98)

Operator command (single, no manual step after startup):

```
cd submodules/helix_qa
# infra: HelixLLM/Ollama endpoint + provider keys from .env (the
    only pre-test bootstrap, §11.4.98(B))
./bin/helixqa autonomous \
  --banks banks/helixllm_llm.yaml,banks/
    helixllm_vision.yaml,banks/helixllm_imagegen.yaml,\
banks/helixllm_videogen.yaml,banks/
    helixllm_translation.yaml,banks/
    helixllm_embeddings_rag.yaml,\
banks/helixllm_stt.yaml,banks/helixllm_ocr.yaml,banks/
    helixllm_providers.yaml \
  --platform all --conduit-dir qa-results/helixllm_<run-id>/
    conduit \
  --evidence-dir qa-results/helixllm_<run-id>
# live-watch in a second pane (real-time §11.4.116 scoreboard):
./bin/helixqa-conduit-monitor --stream qa-results/helixllm_<run-
    id>/conduit/events.jsonl
```

Acceptance criteria (all must hold): 1. The session is **fully self-driving** end-to-end — after key/endpoint bootstrap, zero human action; re-runnable at -count=3 with self-cleaning state (§11.4.98(C)). 2. The conductor tails events.jsonl and sees, per capability, challenge_start → llm_call/vision_call → evidence_captured(evidence_path=..._verdict.json) → challenge_verdict(PASS) — a live, greppable trace. 3. **Every PASS’s verdict artefact exists, is non-empty, and satisfies its content assertion** (the ContentAssertingResolver already enforced this; the operator re-checks a sample by opening the JSON). 4. **The self-validation cases PASS and their paired mutations FAIL** in the meta-test run (make test in submodules/helix_qa + the golden-bad mutation sweep) — proof the oracles can’t bluff. 5. Curated evidence (verdict JSONs, sample images/video/recordings, the conduit stream + final status

snapshot, a rendered summary) is committed under **docs/qa/<run-id>/** (§11.4.83), one subdir per capability, at release-prep only (raw corpus git-ignored per §11.4.128). 6. The final `Status.FinalVerdict` is `PASS` with `Counts` showing every capability's cases `GREEN` or honestly `SKIP/OPERATOR-BLOCKED` with reasons — **no capability silently absent** (§11.4.118 enumerated coverage).

Part 6 — Top 3 Risks

1. **HelixLLM contract is UNCONFIRMED (§11.4.6).** Endpoint paths, request/response schemas, and which capabilities actually ship were not found in-tree. Every `http/dispatches_to/json:` path below is a placeholder — **implementation is blocked until the real HelixLLM API surface is confirmed**; guessing it would itself be a bluff. *Mitigation:* first task of implementation = read/confirm the HelixLLM service spec, fill the `#CONFIRM` values.
2. **Analyzer-is-the-bluff risk.** A metric analyzer (CLIP-score, COMET, WER, faithfulness) that is miscalibrated or lenient `PASSes` broken output — the §11.4.107(10) failure mode. *Mitigation:* the mandatory golden-good/golden-bad + paired-mutation self-validation on **every** analyzer is non-negotiable; thresholds calibrated on committed fixtures, never literature-copied.
3. **Non-determinism + threshold flakiness.** LLM/VLM/gen outputs vary run-to-run; a too-tight assertion flakes, a too-loose one bluffs. *Mitigation:* prefer objectively-checkable prompts (exact-match, entity-preservation, fprobe frame count) over LLM-judge where possible; for genuinely fuzzy metrics use metamorphic relations (back-translation symmetry, relative-order sim, golden-bad separation) and `-count=3` determinism proof (§11.4.50); temperature pinned low for correctness banks.

Report path: `/home/milos/Factory/projects/tools_and_research/helix_code/docs/research/07.2026/12_helixqa_testing/12_helixqa_testing.md`

Sources verified 2026-07-06

- LLM endpoint / load / streaming: <https://gatling.io/blog/load-testing-an-llm-api> · <https://github.com/ray-project/llmperf> · <https://loadforge.com/guides/ai-llm/load-testing-llm-apis-streaming-non-streaming> · <https://blog.prem.ai.io/load-testing-llms-tools-metrics-realistic-traffic-simulation-2026/> · <https://www.truefoundry.com/blog/llm-locust-a-tool-for-benchmarking-llm-performance>
- VLM evaluation: <https://arxiv.org/pdf/2311.14580> (LLMs as Automated Aligners) · <https://www.nature.com/articles/s41598-026-55179-4> (VLMs for

- automated quality control) · <https://labelyourdata.com/articles/machine-learning/vision-language-models> · <https://arxiv.org/pdf/2508.19294>
- Image/video-gen validation: <https://labelbox.com/guides/a-comprehensive-approach-to-evaluating-text-to-image-models/> · <https://imagebench.ai/learn/prompt-fidelity> · <https://ffmpeg.org/ffprobe.html> · <https://ottverse.com/extract-frame-count-using-ffprobe-ffmpeg/>
 - Translation quality: https://grokipedia.com/page/Evaluation_of_machine_translation · <https://arxiv.org/pdf/2508.18549> (COMET-poly) · https://direct.mit.edu/tacl/article/doi/10.1162/tacl_a_00711/125277/Conformalizing-Machine-Translation-Evaluation
 - RAG / embeddings: <https://langcopilot.com/posts/2025-09-17-rag-evaluation-101-from-recall-k-to-answer-faithfulness> · <https://www.confident-ai.com/blog/rag-evaluation-metrics-answer-relevancy-faithfulness-and-more> · <https://www.getmaxim.ai/articles/complete-guide-to-rag-evaluation-metrics-methods-and-best-practices-for-2025/> · <https://labelyourdata.com/articles/llm-fine-tuning/rag-evaluation>
 - STT / OCR: <https://mlcommons.org/2025/09/whisper-inferencev5-1/> · <https://whisperapi.com/word-error-rate-wer> · <https://towardsdatascience.com/evaluating-ocr-output-quality-with-character-error-rate-cer-and-word-error-rate-wer-853175297510/> · <https://www.handwritingocr.com/blog/character-error-rate-explained>
 - HelixQA source (read 2026-07-06): `submodules/helix_qa/{README.md, pkg/testbank/schema.go, pkg/testbank/content_evidence.go, pkg/testbank/dispatch.go, pkg/conduit/{event,emit,monitor}.go, pkg/vision/{llm_ollama,ocr_tesseract,detector,diff}.go, pkg/recordingqa/recordingqa.go, banks/atmosphere_subtitles.yaml, cmd/}`